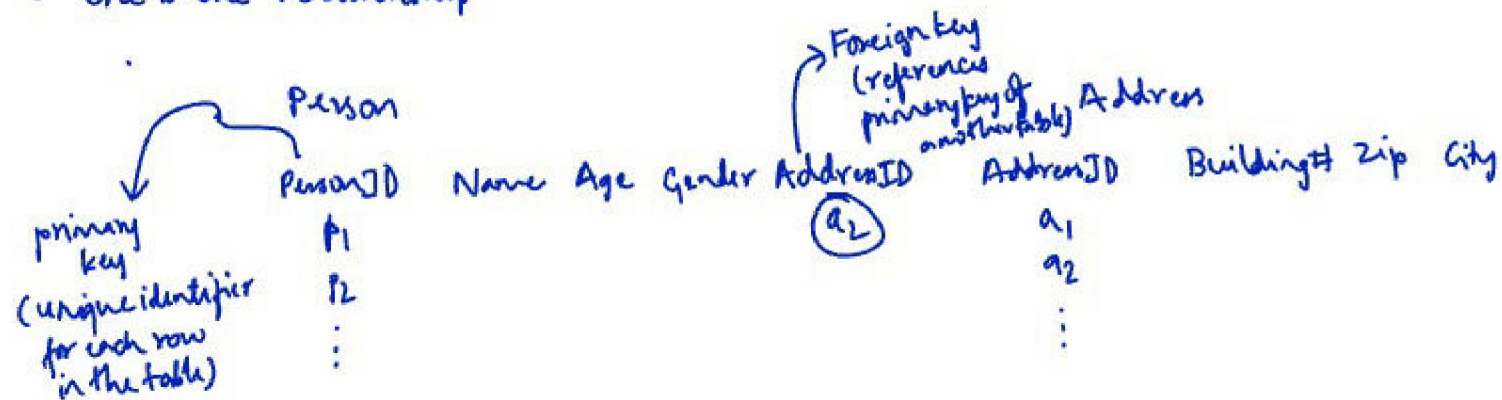


- One to one relationship



- One to many relationship

- Each professor belongs to exactly one dept
- Each dept can have multiple professors

Diagram illustrating a one-to-many relationship between Prof and Dept tables.

Prof Table:

ProfID	Name	Salary	deptID
p1	n1	s1	d1
p2	n2	s2	d2
p3	n3	s3	d1

Annotation: **foreign key** points to deptID in the Prof table.

Dept Table:

DeptID	DeptName	Dept Building #
d1	dn1	db1
d2	dn2	db2

- Many to Many relationship

- Students & classes

Diagram illustrating a many-to-many relationship between Students and Classes tables.

Students Table:

StudentID	Name	Age
s1	n1	a1
s2	n2	a2
s3	n3	a3

Classes Table:

ClassID	Name	DeptID
c1	cn1	d1
c2	cn2	d1

StudentsClasses Table (Join Table):

StudentID	ClassID
s1	c1
s1	c2
s2	c1
s2	c2

8 bits = 1 byte

Kilo-1000

2^{10} Bytes = 1024 Bytes $\sim 10^3$ Bytes = 1 KB

$2^{10} \sim 10^3$

2^{20} Bytes: $\sim 10^6$ Bytes = 1 MB

$2^{20} \sim 10^6$

2^{30} Bytes: $\sim 10^9$ Bytes = 1 GB

Mega- 10^6

2^{40} Bytes: $\sim 10^{12}$ Bytes = 1 TB

Giga- 10^9

2^{50} Bytes: $\sim 10^{15}$ Bytes = 1 PB

$$(a^5)^2 = a^{10}$$

$$a^5 \times a^5 = a^{10}$$

$$(a^m)^n = a^{mn}$$

$$a^m \times a^n = a^{m+n}$$

$$\frac{a^m}{a^n} = a^{m-n}$$

D R A M A
 $S_5 | S_4 | S_3 | S_2 | S_1$ Stages

Pipelining

- divide instructions into stages
- serve multiple instr at the same time (different stages)
- increases throughput

Cache

Registers - 32 registers

L1 Cache (each 64bit/32bit)

L2 Cache

L3 Cache

Main Memory (RAM)

Disk

Cost $\uparrow$
Capacity $\downarrow$
Speed $\uparrow$

$a++$ } critical section

1. load a from mem into reg.
2. inc reg
3. store the value in memory

T_1
1000 $\rightarrow$ load a
inc reg

store

a 1001

T_2

load $a \rightarrow 1000$
inc reg
store

1 ns

10^0

100 ns

10^2

10^5

Cache:

- Temporal locality
- Spatial locality

L_1 cache 1ns
 Branch misprediction 3ns
 L_2 cache ref 4ns
 mutex lock/unlock 17ns

Main memory ref 100ns

Compressing 1KB - 2000ns = 2µs

SSD read - 16000ns = 16µs
(random)

Read 10^6 Bytes = 1MB = 3000ns = 3µs
sequentially from memory

Round trip in the same data = 500,000ns = 500µs
center

Reading 1MB sequentially = 49000ns = 49µs
from SSD

Disk seek: 2000000ns = 2000µs = 2ms

packet roundtrip CA to Ned = 150ms
 = 150000µs
 = 150000000ns

$1\text{GHz} \rightarrow 10^9 \text{ clock cycles/second}$
 Hertz - 1/second frequency
 Giga - 10^9

$\Rightarrow$ Each clock cycle takes $1/10^9 \text{ sec}$

10^9 - Giga
 10^6 - Mega
 10^3 - kilo

10^{-3} - milli
 10^{-6} - micro
 10^{-9} - nano

Availability Numbers:

• 4 nines $\Rightarrow 99.99\%$

• 6 nines $\Rightarrow 99.9999\%$

1 Day = 1440 min
 = 86400s

Availability	Downtime per day	Downtime per year
99%	14.4 min	3-65 days
⋮	⋮	⋮

• Assumptions:

- 300 million monthly active users
- 50% of users daily
- 2 tweet per day
- 10% of tweets contain media
- Data is stored for 5 years

• Queries per second

- 150 million user per day
- 300 million tweets per day
- $\frac{300 \times 10^6}{86400} \approx 3500$ QPS

$$= \frac{3000}{864} \times 1000$$

$$\approx 3.5 \times 1000$$

$$\frac{99987}{9.1} = \frac{100000}{10}$$

$$\frac{0.987 \times 10^5}{1} = \frac{1 \times 10^5}{1}$$

$$864 \times 3 \approx 2600$$

• Peak QPS = 2×3500 QPS = 7000 QPS

- Media storage:
- | | |
|----------|-----------|
| tweet Id | 64 Bytes |
| text | 140 Bytes |
| media | 1MB |

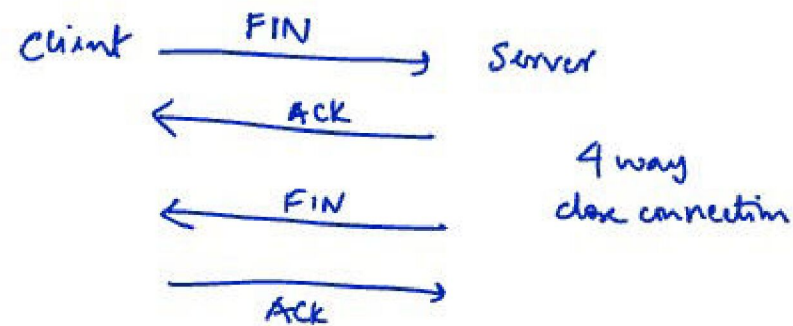
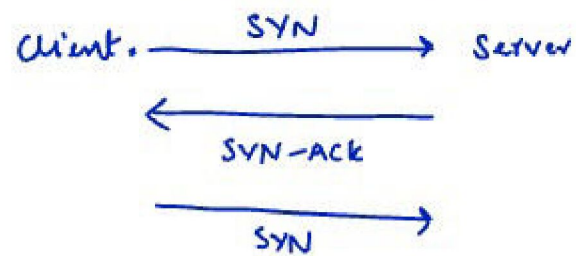
• storage per day: 30 million tweets have media = $30 \text{ million} \times 1 \text{ MB} = 3 \times 10^7 \times 10^6 \text{ Bytes}$
 $= 3 \times 10^{13} \text{ bytes} = 30 \text{ TB}$

• Across 5 years: $30 \text{ TB} \times 5 \times 365 = 3 \times 365 = 1100$
 $30 \times 365 = 11000$
 $\times 5 = 55000$
 $= 55000 \text{ TB}$
 $= 55 \text{ PB}$

- Layered model

- ★ Application ✓ HTTP
Presentation
Session
- ★ Transport ✓ TCP, UDP, QUIC
- ★ Network layer ✓ IP
Data link layer
Physical layer

- TCP 3way handshake



- Encryption

- Symmetric key encryption
 - encrypt & decrypt with the same secret key
 - fast
- Asymmetric key encryption
 - public-private key
 - encrypt with public key
 - decrypt with corresponding private key
 - slower
- Secret key (symmetric) shared using asymmetric key encryption scheme

• Network layer

$$2^{10} \sim 10^3$$

- IP
- Two versions of IP addresses

- IPv4 - 4 byte $\rightarrow 2^{32} \sim 4 \times 10^9$ (4 Billion)
- IPv6 - 16 byte

private
& public IP

- 10.0.0.0/8
- 192.168.0.0/16

private addresses

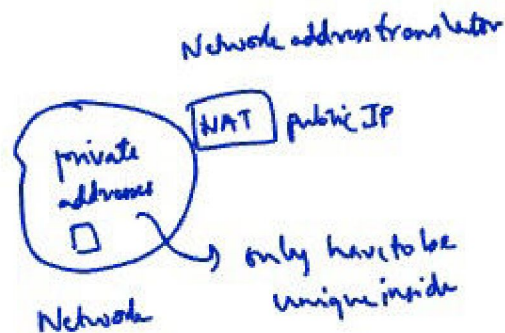
- 18.0.0.0/8 $\rightarrow$ # hosts we can have in this network = 2^{24}

$\rightarrow$ represents the # bits to represent the network

$\Rightarrow$ 24 bits - represent the host on the network

$$2^8 = 256$$

0 to 255



• Transport layer

- 4 tuple $\rightarrow$ src IP
src port
dest IP
dest port

• TCP

- Connection oriented
- Reliable delivery
- Flow control
- Congestion control
- 20-60 bytes

• UDP

- Connectionless
- No guarantee of delivery
- No ordering
- Lower latency
- 8 bytes header

- Application layer protocols

- HTTP / HTTPS

- REST: Architectural style

- Resources (Nouns)

- Methods (verbs): GET, POST, PUT, PATCH, DELETE

- /users/{id} GET

- /users POST

- /users/{id}/posts GET

method

headers: key-value pairs for metadata

can request for a certain compressed format but server might not respond with that format
- backward compatibility

Response:

status code

2xx - successful
4xx - client side error
5xx - server side error

200 OK
201 Created
404 Not found
401 Unauthorized
403
500
502

- GraphQL

- RPCs

- userService.updateData(..)

- gRPC

- Protobufs

```

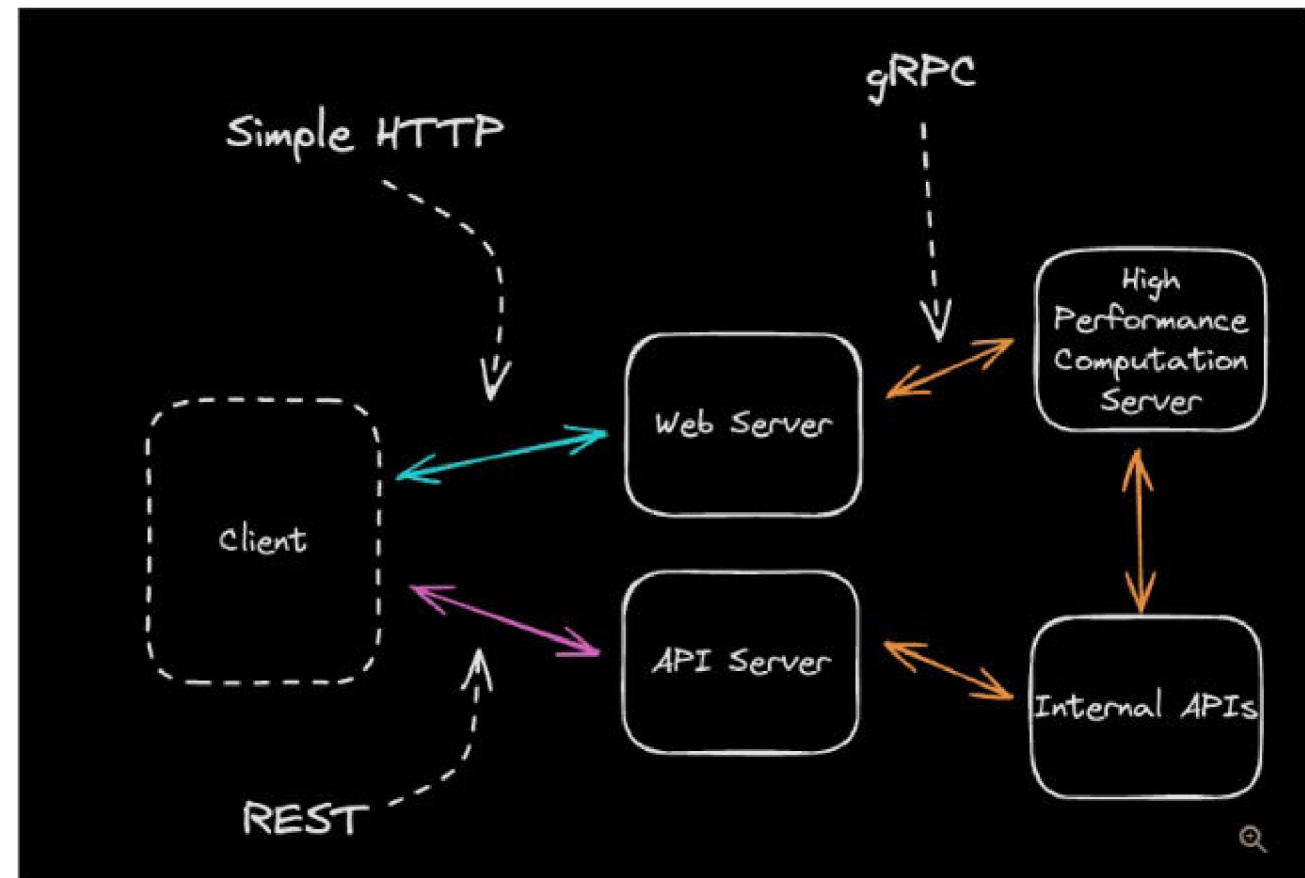
user {
  id
  name
  age
}
    
```

- JSON: 40 bytes

- Serialized: 15 bytes

Query param

?page=4&offset=10



Server Side Events (SSE) - push based

Websockets

- Bidirectional communication with high frequency updates with low latency
- think in terms of messages exchanged instead of req-resp

WebRTC (UDP)

- peer to peer communication
- collaborative document editing, audio-video calls

Scaling

- Vertical scaling
- Horizontal scaling

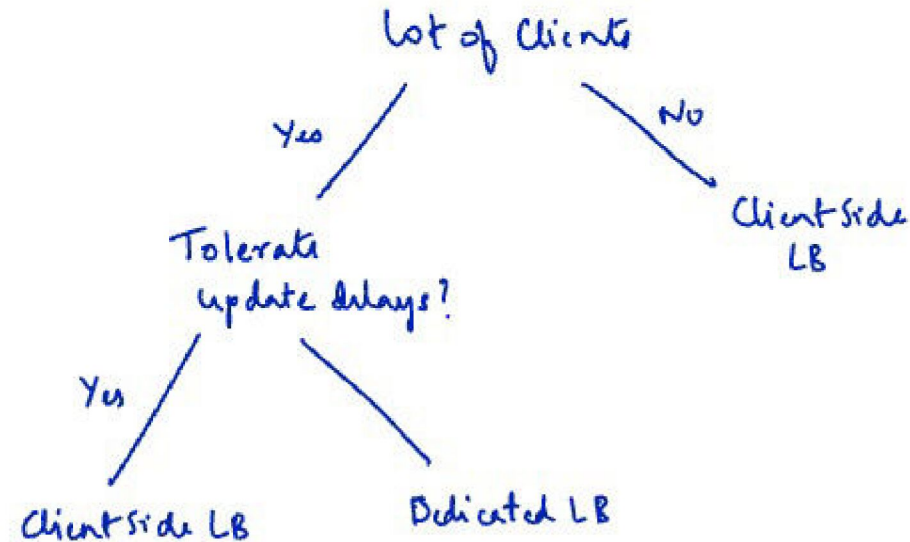
Load Balancer

- Client Load Balancer
- A dedicated load balancer

Eg: Redis cluster
DNS



To prevent SPOF
in a LB, we can have
multiple dedicated LBs
& have DNS rotate among
the IPs



- Client LB works very well for internal microservices
- built into gRPC

- Dedicated load balancer

- If we don't want our clients to refresh the list of servers

or

If we don't want to give extra info about servers, use a dedicated LB

- Extra network hop

but we get to decide how to route

- Can occur at two levels

- L4 load balancers (Transport layer)

- L7 load balancer (Application layer)



- examines actual data & intelligent routing decisions

- LB is end point for first TCP connection

- make decisions based method, headers, cookies, URL etc

- CPU intensive (examine packets)

- flexible

- suited for HTTP traffic

- Doesn't look inside packets (application data)

- It's like randomly selecting a server

- Works as if there is a direct TCP connection between client & server

- fast & efficient

- raw performance is priority

- use when persistent connection is required

• Eg: websockets

- Health checks & fault tolerance

- LB can do health checks periodically to figure out which servers are up

- Health checks can be configured



TCP request
(200 ⇒ OK
5xx ⇒ down)

- Load balancing algos:

- Round robin
 - Randomly decide
 - Least connections
 - Least response time
 - IP hash
- } work best for stateless applications
- useful for persistent connections to prevent accumulation on one server
- } stateful applications

- Dedicated LB

hardware LBs — physical devices — can handle 100s of millions req per sec

software LBs — HAProxy, Nginx etc — order of millions req per sec

cloud LBs — AWS, GCP

- Regionalization & latency

- Light in fibre optic cables travels at $\frac{2}{3}$ rd of light speed
~ 200 000 km/s

- Distance New York & London — 5600 km between

- Round trip for light itself — $\frac{56 \text{ ms}}{2}$

↓
theoretical minimum

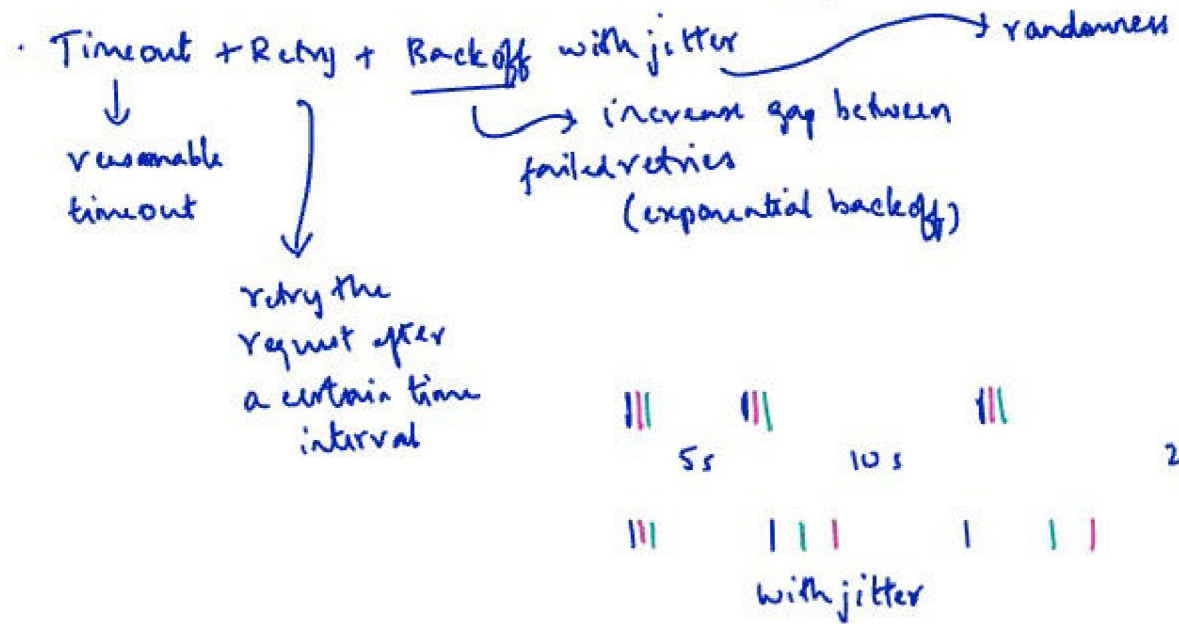
8900 km
⇒ round trip's
theoretical
min is 89 ms

• Regionalization

1. Keep server & database close
 2. Keep both of them close to the user
- Have multiple data centers in a single region & repeat this pattern across multiple regions
 - CDNs
 - static data & media
 - cached computationally expensive data
 - Regional partitioning
 - Uber

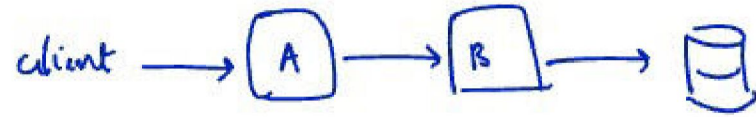
• Handling failures

- Server failures
- Network failures - Don't assume that network is reliable (dropped packets, corrupted data etc)



- To prevent side effects in retries, we make the requests idempotent
 - Idempotency Key: to figure out if the same request

• Circuit Breakers



Cascading failure if DB is down.

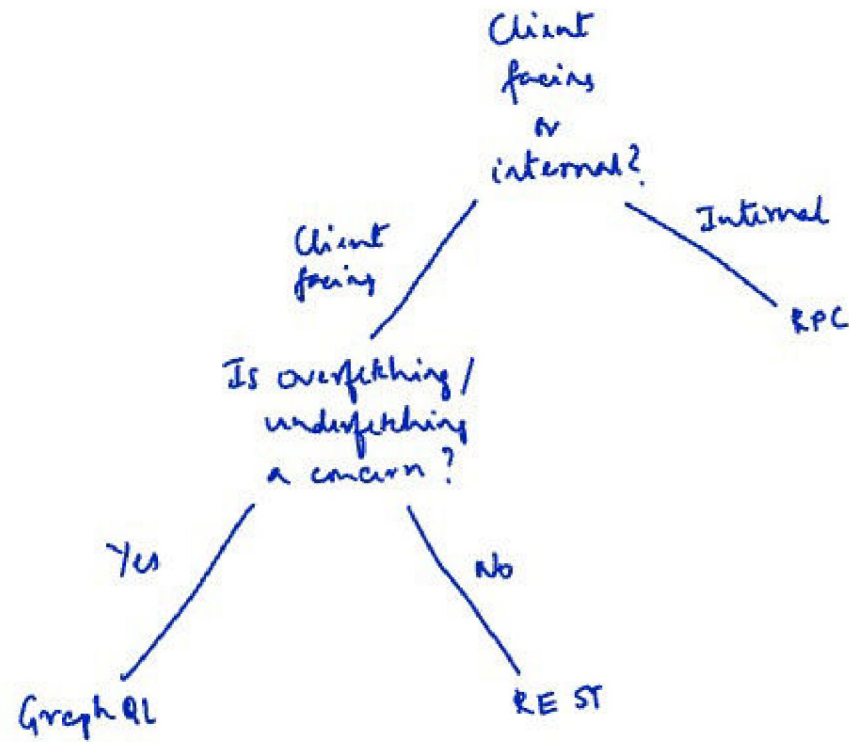
• Circuit breakers

API Design:

- How clients interact with our system

- Three main options

- REST → default choice (90% of the time this is the correct choice)
- GraphQL
- RPC



• REST

- Core entities are your resources (nouns)

- Eg: events, tickets, venues, bookings

- Get all events: GET /events

- Get a particular event: GET /events/{id}

- Create a new booking for an event: POST /events/{id}/bookings

POST /bookings

} plural nouns - always

• Relationships between resources

1. nest resources with path parameters

/events/{id}/tickets



path parameter

2. If we want users to directly interact with tickets - filter, sort etc, use query parameters

/tickets? k₁=v₁ & k₂=v₂ & k₃=v₃

/tickets?eventId=123

- GET - retrieve

(without any side effect)

- idempotent

- POST - create new resources

- not idempotent

- PUT - replace entire resource

- idempotent

- PATCH - update part of a resource

- not idempotent (eg: increment)

- DELETE - removing a resource

- idempotent (first delete - 204
subsequent - 404
deletes)

- Three options to pass data to the API
 - path parameters - eg: resource id
 - query parameters - for optional parameters
 - filters
 - pagination
 - request body - data structures
 - payload

- Return data (response)
 - status code ✓
 - 2xx
 - 4xx
 - 5xx
 - response body

• GraphQL

- client decides the shape of data
- use cases:
 - different kinds of devices
 - constantly changing frontend
 - separate independent teams for frontend & backend

• complications

- query parsing, schema validation
- sophisticated caching challenges

• N+1 problem

- Event {
 - venue: Venue!
 - ...

- 100 events - get all of them
 - 1 query

SELECT * FROM events LIMIT 100

- 100 venues' data - 100 queries for each of the 100 venue ids

100 + 1 = 101 queries

- batching / dataloader strategies to handle this